CS527 Project Proposal: LLM-Enhanced Fault Localization

Zijie (Sam) Qiu (zijieq2) · Yuxuan (Steven) Li (yuxuan43) · Peiling (Alina) Wu (peiling4)

1. Target problem and motivation

When a test fails, the slow part of debugging is not writing the fix but finding which method is wrong. Fault localization automates that search, and the classical solution is spectrum-based fault localization (SBFL): record which methods each test executes, then score a method by how strongly its execution correlates with failure.

SBFL is cheap and works on any project with a test suite, but it only counts executions. It cannot tell that a three-line accumulator loop is obviously correct while the one-line method above it divides two integers. So everything on the failing execution path scores highly, and the truly buggy method often lands outside the top few positions, where developers stop looking. Large language models have the opposite profile: they read code semantically and can explain why a line contradicts an error message, but they cannot scan a hundred thousand methods per bug, and feeding a whole project into a prompt is neither affordable nor accurate. Neither technique is sufficient alone, which is what makes the combination worth building.

2. Approach design

We build a hybrid pipeline in which a classical analysis narrows the search space and an LLM makes the semantic judgment inside it: **failing test** → **coverage per test** → **SBFL ranking (top 20 methods)** → **context builder** → **LLM re-ranker** → **rank fusion**.

(1) Coverage. Run the test suite on the buggy version and record which lines each test executes, plus its pass or fail outcome. **(2) SBFL.** Score every line with Ochiai, take each method's score as the maximum over its lines, and keep the top 20 candidates. **(3) Context builder.** For each candidate, assemble the method body, the failing test source, the assertion message, and the stack trace, staying inside a fixed token budget. **(4) LLM re-ranker.** Present all 20 candidates in one prompt and ask for a ranked list with a one-line justification for the top three. Candidate order is shuffled on every sample to control position bias, and we take five samples and average the ranks. **(5) Fusion.** Combine the LLM rank with the original SBFL rank by weighted rank fusion, so a confident statistical signal is not discarded when the model is unsure.

The pipeline is Python; Defects4J and the Java projects are driven through the command line, and method boundaries are extracted with a Java parser. Every model response is cached on disk so that reruns are free and results are reproducible.

3. Evaluation design

Subjects. Defects4J, a benchmark of real Java bugs, each shipped with a buggy version, a fixed version, the failing tests, and the developer's patch. We use the Lang, Math, Chart, and Time projects (roughly 220 bugs) and exclude bugs whose fix only adds new methods, since no buggy method then exists to rank. Closure and Mockito are excluded initially because their files are too large to prompt affordably.

Ground truth. The methods touched by the developer's patch, so no manual judging is required. **Metrics.** Top-1, Top-3, and Top-5 accuracy, mean reciprocal rank, and tokens plus dollars per bug. **Baselines.** Ochiai alone; DStar alone, a second formula computed from the same coverage data; and an LLM without SBFL, where candidates come from the classes named in the stack trace. **Ablations.** Remove the stack trace, the test source, or the error message; vary the candidate count (10, 20, 50); compare one sample against five; compare a cheap model against a strong one; sweep the fusion weight, tuned on one project and tested on the others.

Leakage check. Defects4J bugs predate 2019 and are plausibly memorized by the models we use. We therefore repeat the evaluation on a small set of post-2023 Java bugs from GitBug-Java and report both numbers.

4. Expected outcome

We expect the re-ranker to raise Top-1 accuracy over Ochiai by a clear margin on the same bug set, with a smaller gain at Top-5, where SBFL already performs reasonably. We commit to reporting the result either way, including the cost per bug, so that a modest gain at low cost is still a reportable finding.

Deliverables: an open-source pipeline that turns a Defects4J bug ID into a ranked list of suspect methods with justifications; a results table covering four projects, three baselines, and the ablations above; and the leakage comparison on recent bugs. If time allows, we will add an agentic variant in which the model can read callers and related files before ranking, and compare it against the fixed-context version on accuracy and cost.